# VIT BHOPAL UNIVERSITY

**Course:** CSE0002 – OPEN SOURCE SOFTWARE

**Project Title:** The Open Source Audit: Python Ecosystem

**Academic Program:** Integrated M.Tech in Artificial Intelligence and Machine Learning

**Student Name:** Upma Shukla

**Registration Number:** 24MIM10029

**Slot:** B22

**Date of Submission:** 29-03-2026

# TABLE OF CONTENTS

1. Introduction
2. PART A: Origin and Philosophy ·
   A1. The Problem and Origin Story ·
   A2.The License: An In-depth Analysis of the PSF License
   A3. The Ethics of Open Source Software
3. PART B: Linux Footprint and Technical Documentation
4. PART C: The FOSS Ecosystem and Community
5. PART D: Open Source vs. Proprietary: A Comparative Analysis
6. PART E: Shell Script Documentation (The Audit Tools) ·
   Script 1: System Identity Report ·
   Script 2: FOSS Package Inspector ·
   Script 3: Disk and Permission Auditor ·
   Script 4: Log File Analyzer ·
   Script 5: Manifesto Generator
7. PART F: Reflection and Conclusion
8. References

# 1. Introduction

The modern digital world is built upon a foundation of collaboration and shared knowledge. While proprietary software dominates certain consumer markets, the invisible infrastructure of the internet, advanced data science, and artificial intelligence is powered by Open Source Software (OSS). The philosophy behind OSS is a paradigm shift in how technology is created, championing transparency, community-driven development, and the fundamental right of a user to understand and modify their tools.

This project, titled "The Open Source Audit," is a structured investigation into one of the most successful and widely utilized programming languages in modern computing: Python. Python has become the industry standard for backend development, automation, and prominently, Artificial Intelligence and Machine Learning. Its success is not an accident; it is a direct result of its community-driven, open-source development model.

The primary objectives of this audit are threefold:

1. **To Explore the Origin and Philosophy:** To understand the historical context that led to Python's creation, analyzing the problem it solved and examining its permissive Open Source license (the PSF License).
2. **To Analyze the Technical Footprint:** To document the installation, configuration, and runtime behavior of Python on a Linux system, identifying its binaries, cache directories, and package management mechanisms.
3. **To Create Practical Audit Tools:** To develop a suite of custom shell scripts that demonstrate the power of the Linux command line for system auditing, package verification, and log analysis.

Every professional line of code written today rests on the shoulders of open-source giants. This report documents a comprehensive analysis of the Python programming language as a case study in open-source infrastructure.

---

# 2. PART A: Origin and Philosophy

## A1. The Problem and Origin Story

Python was developed in the late 1980s by Guido van Rossum at the Centrum Wiskunde & Informatica (CWI) in the Netherlands. At that time, there was a need for a high-level scripting language that could effectively bridge the gap between low-level system programming languages such as C and simple shell scripting tools.

The Amoeba operating system required a scripting solution that was both easy to understand and sufficiently powerful to manage system-level tasks. Instead of keeping the language proprietary, van Rossum made the decision to release Python's source code publicly in 1991.

This open distribution encouraged global collaboration, allowing developers to contribute to its growth. Over time, this collaborative model significantly expanded Python's standard library and capabilities, ultimately establishing it as one of the most widely used languages in fields such as Artificial Intelligence and data science.

## A2. The License:

**An In-depth Analysis of the PSF License** Unlike the viral GNU General Public License (GPL), Python is distributed under the Python Software Foundation (PSF) License. This is an OSIapproved, permissive open-source license.

- **Permissive Nature:** It allows users to study, modify, and distribute the software, but unlike "copyleft" licenses, it permits developers to include Python in proprietary, closedsource applications without forcing them to release their own source code.
- **Corporate and Academic Adoption:** This permissiveness is a primary reason Python was so rapidly adopted by both academic institutions and major tech corporations, fueling the massive growth of ML frameworks like TensorFlow and PyTorch.

## A3. The Ethics of Open Source Software

The success of Python highlights the ethical superiority of transparency in software. In AI and ML, where algorithms can contain biases or "black box" logic, the ability to inspect the underlying code and mathematical libraries (like NumPy) is crucial for ethical computing. Furthermore, Python demonstrates the concept of "standing on the shoulders of giants"; no single researcher builds a neural network from absolute scratch. They rely on the shared, open-source ecosystem that the Python community meticulously maintains.

---

# PART B: Linux Footprint and Technical Documentation

To conduct this technical audit, Python was analyzed on a standard Linux distribution.

**Installation Method:** Python is fundamentally integrated into most modern Linux distributions. It is managed using the system's native package manager (apt).

- **Command:** sudo apt update && sudo apt install python3 -y
- **Rationale:** Centralized software management makes it easy to install and update while maintaining a consistent system state.

**Key Directories and File Locations:**

Following the Filesystem Hierarchy Standard (FHS), the audit revealed:

- **Binary Executable (/usr/bin/python3):** The main interpreter file that executes when invoked from the terminal.
- **System Libraries (/usr/lib/python3.x/):** Contains the core standard library modules that ship with Python.
- **Package Cache (~/.cache/pip/):** A critical directory for AI/ML students. As heavy libraries (like Pandas or Scikit-learn) are downloaded, this cache grows, requiring periodic audits to prevent disk exhaustion.

---

# PART C: The FOSS Ecosystem and Community

Python is not merely a language; it is a central node in a vast open-source ecosystem.

**The Dependency Tree (PIP):**

Python's real power comes from its package installer, pip, which connects developers to the Python Package Index (PyPI). A developer writing a machine learning script does not write the math from scratch; they import open-source libraries.

- **NumPy & Pandas:** For data manipulation and scientific computing.
- **Scikit-Learn & PyTorch:** Advanced FOSS libraries for building predictive models.

**Community Structure:** The project is governed by the Python Software Foundation (PSF), a non-profit organization ensuring the project's goals remain focused on its users. The Python Enhancement Proposal (PEP) system allows any community member to suggest and debate new features for the language, embodying true democratic software development.

---

# PART D: Open Source vs. Proprietary: A Comparative Analysis

To fully appreciate Python, it must be compared to proprietary environments like MATLAB, which is heavily used in traditional academic engineering.

| Dimension | Python (Open Source) | MATLAB (Proprietary) |
|---|---|---|
| **Cost** | **Free (Gratis):** No licensing fees. Can be installed on unlimited cloud servers or local machines. | **High Cost:** Requires expensive institutional or individual licenses to operate legally. |
| **Ecosystem** | **Extensive:** Driven by PyPI, with instant, free access to the world's most advanced AI/ML libraries. | **Closed:** Toolboxes must be purchased individually. Development is restricted to MathWorks engineers. |
| **User Freedom** | **Full Right to Modify:** Users can dig into the source code of any library to see exactly how algorithms are implemented. | **Strictly Prohibited:** Algorithms are often black-boxed, preventing researchers from auditing the underlying math. |

**Verdict:** For modern computer science, particularly Artificial Intelligence, Python is overwhelmingly the superior choice. Its open nature prevents vendor lock-in, ensures reproducibility in academic research, and allows students to utilize enterprise-grade tools at zero cost.

---

# PART E: Shell Script Documentation (The Audit Tools)

- **Script 1: System Identity Report (script1_identity.sh)**:

  Extracts system variables (Distro, Kernel, User) to document the environment before ML deployment.

```
#!/bin/bash
# Script 1: System Identity Report
# --- Variables ---
STUDENT_NAME="Upma Shukla"
SOFTWARE_CHOICE="Git"

# --- System Info ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime -p)
DATE=$(date)
DISTRO=$(lsb_release -d | cut -f2)

# --- Display ---
echo "========================================"
echo "   Open Source Audit — $STUDENT_NAME"
echo "========================================"
echo "Software : $SOFTWARE_CHOICE"
echo "Distro   : $DISTRO"
echo "Kernel   : $KERNEL"
echo "User     : $USER_NAME"
echo "Home Dir : $HOME_DIR"
echo "Uptime   : $UPTIME"
echo "Date     : $DATE"
echo "License  : GNU General Public License (GPL)"
echo "========================================"
```

- **Script 2: FOSS Package Inspector (script2_inspector.sh)**:

  Uses command -v python3 to verify the installation and version of the Python interpreter, validating the open-source environment.

```
#!/bin/bash
# Script 2: FOSS Package
Inspector

PACKAGE="git"   # you can
change if needed

# Check if package is
installed
if dpkg -l | grep -q
$PACKAGE; then
    echo "$PACKAGE is
installed."
    dpkg -l | grep
$PACKAGE | grep "^ii" |
awk '{print "Version:",
$3}'
else
    echo "$PACKAGE is NOT
installed."
fi

# Case statement
(completed TODO)
case $PACKAGE in
    httpd)  echo "Apache:
the web server that built
the open internet" ;;
    mysql)  echo "MySQL:
open source at the heart
of millions of apps" ;;
    git)    echo "Git:
version control system for
open collaboration" ;;
    vlc)    echo "VLC:
open source media player
that plays everything" ;;
    firefox) echo
"Firefox: open browser
focused on privacy and
freedom" ;;
esacesac
```

- **Script 3: Disk and Permission Auditor (script3_auditor.sh)**:

  Analyzes system directories and specifically targets the ~/.cache/pip directory to monitor storage utilization by data science packages.

```
#!/bin/bash
# Script 3: Disk and Permission Auditor

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"
echo "----------------------"

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
        echo "$DIR => Permissions: $PERMS | Size: $SIZE"
    else
        echo "$DIR does not exist on this system"
    fi
done

echo "----------------------"

# Added TODO part (for Git config directory)
CONFIG_DIR="$HOME/.git"

if [ -d "$CONFIG_DIR" ]; then
    PERMS=$(ls -ld $CONFIG_DIR | awk '{print $1, $3, $4}')
    echo "Git Config Dir ($CONFIG_DIR) => Permissions: $PERMS"
else
    echo "Git config directory not found"
fi
```

- **Script 4: Log File Analyzer (script4_analyzer.sh)**:

  Uses a memory-efficient while-read loop to parse /var/log/syslog for keywords like "error", demonstrating essential troubleshooting techniques for server-side Python applications.

```
#!/bin/bash
# Script 4: Log File Analyzer

LOGFILE=$1
KEYWORD=${2:-"error"}
COUNT=0

if [ ! -f "$LOGFILE" ]; then
    echo "Error: File $LOGFILE not found."
    exit 1
fi
```

```
while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD";
then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT
times in $LOGFILE"

# Show last 5 matching lines
echo "Last 5 matching lines:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

- **Script 5: Manifesto Generator (script5_manifesto.sh)**:

  Interactively prompts the user to define their core open-source values and AI aspirations, utilizing file redirection to generate a customized .txt declaration.

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator

echo "Answer three questions to generate your
manifesto."
echo ""

read -p "1. Name one open-source tool you use every
day: " TOOL
read -p "2. In one word, what does 'freedom' mean to
you? " FREEDOM
read -p "3. Name one thing you would build and share
freely: " BUILD

DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

echo "" > $OUTPUT
echo "Open Source Manifesto" >> $OUTPUT
echo "Date: $DATE" >> $OUTPUT
echo "" >> $OUTPUT

echo "I use $TOOL every day, and it represents the
idea of $FREEDOM." >> $OUTPUT
echo "I believe in sharing knowledge openly." >>
$OUTPUT
echo "One day, I want to build $BUILD and share it
freely with the world." >> $OUTPUT

echo ""
echo "Manifesto saved to $OUTPUT"
cat $OUTPUT
```

---

# PART F: Reflection and Conclusion

This project provided a deep and practical dive into the world of Open Source Software using the Python programming language as a concrete case study.

The key takeaways from this audit are:

- **Philosophical:** Permissive licenses like the PSF license foster massive industry adoption, proving that open collaboration accelerates technological breakthroughs—especially in the rapidly evolving field of AI/ML.
- **Technical:** The Linux environment, with its transparent filesystem and powerful command-line tools, is the ideal platform for managing programming environments. The

five shell scripts developed serve as functional tools for system administration and environment management.

- **Practical:** Python stands as a testament to the success of the open-source model. Its unmatched ecosystem of libraries makes it the undisputed leader in modern software engineering and data science.

Ultimately, this project reinforced that open source is about freedom, transparency, and community. As a student of Artificial Intelligence and Machine Learning, understanding how to navigate, audit, and contribute to this Linux and Python ecosystem is a foundational professional responsibility.

# 8. References


1. Free Software Foundation. (n.d.). *What is free software?* Retrieved from https://www.gnu.org/philosophy/free-sw.html
2. GNU Project. (n.d.). *GNU General Public License, version 2.* Retrieved from https://www.gnu.org/licenses/old-licenses/gpl-2.0.html
3. Open Source Initiative. (n.d.). *The Open Source Definition.* Retrieved from https://opensource.org/osd
4. Shotts, W. (2019). *The Linux Command Line: A Complete Introduction* (2nd ed.). No Starch Press.
5. VideoLAN Organization. (n.d.). *Official VLC Media Player Documentation.* Retrieved from https://www.videolan.org/vlc/
6. Nemeth, E., Snyder, G., Hein, T. R., & Whaley, B. (2017). *UNIX and Linux System Administration Handbook* (5th ed.). Addison-Wesley Professional.
7. Raymond, E. S. (1999). *The Cathedral and the Bazaar.* O'Reilly Media.
8. Ubuntu Documentation. (n.d.). *Apt Package Manager.* Retrieved from https://help.ubuntu.com/community/AptGet/Howto
9. FFmpeg Project. (n.d.). *About FFmpeg.* Retrieved from